

Tab 1

MODULE 1: AUTHENTICATION MODULE (Detailed)

1. Overview

The **Authentication Module** is responsible for verifying user identity and securing access to the system. It ensures that only authorized users can interact with protected resources like sending messages or viewing chats.

This module acts as the **entry point** of the system and is critical for maintaining security and data integrity.

2. Objectives

- Register new users
 - Authenticate existing users
 - Issue secure tokens (JWT)
 - Protect backend routes
 - Maintain session integrity
-

3. Responsibilities

◆ User Registration

- Accept user details (name, email, password)
 - Validate input
 - Hash password before storing
 - Save user in database
-

◆ User Login

- Verify email & password

- Compare hashed password
 - Generate JWT token
-

◆ Token Management

- Create JWT token on login
 - Attach user ID inside token
 - Set expiration (e.g., 7 days)
-

◆ Route Protection

- Middleware verifies JWT
 - Blocks unauthorized requests
-

4. Architecture (Flow)

User → Register/Login → Backend
↓
Password Hashing (bcrypt)
↓
Database (store user)
↓
JWT Token Generated
↓
Client stores token
↓
Every request → token verification

5. Database Schema (User Table)

```
{  
  "userId": "uuid",  
  "name": "Avi",  
  "email": "avi@email.com",  
  "password": "hashed_password",  
  "createdAt": "timestamp"  
}
```



6. Security Design

◆ Password Hashing

- Use **bcrypt**
 - Never store plain passwords
-

◆ JWT (JSON Web Token)

- Signed token containing:
 - `userId`
 - Used for authentication
-

◆ Token Expiry

- Example: 7 days
 - Prevents long-term misuse
-

◆ Secure Headers

- Use HTTPS
 - Prevent token leakage
-



7. API Endpoints

➤ Register

POST `/api/auth/register`

Request:

```
{
  "name": "Avi",
  "email": "avi@email.com",
  "password": "123456"
}
```

➤ Login

POST /api/auth/login

Response:

```
{  
  "token": "jwt_token_here",  
  "userId": "123"  
}
```

➤ Get Current User

GET /api/auth/me

👉 Requires token

🧩 8. Middleware (Very Important)

◆ Auth Middleware

- Extract token from header
- Verify token
- Attach user to request

Example:

```
req.user = decoded.userId;
```

⚠ 9. Edge Cases

- Duplicate email registration
 - Invalid login credentials
 - Expired token
 - Missing token
-

🧠 10. Design Decisions (for interview)

- Used JWT instead of sessions → scalable

- Used bcrypt → secure password storage
 - Stateless authentication → better for APIs
-

11. Resume Explanation

Implemented a secure authentication system using JWT-based stateless authentication, bcrypt password hashing, and middleware-based route protection.

12. Why This Module is Critical

- All other modules depend on it
- Ensures system security
- First impression in backend design

Tab 2

MODULE 2: USER MODULE (Detailed)

1. Overview

The **User Module** manages all user-related data and relationships in the system. It handles user profiles, friend connections, and user discovery.

👉 After authentication, this is the module that defines **who the user is and how they connect with others**.

2. Objectives

- Store and manage user profile data
 - Enable friend connections
 - Fetch user lists for interaction
 - Support search and discovery
-

3. Responsibilities

◆ User Profile Management

- Get user profile
 - Update basic info (name, avatar optional)
-

◆ Friend Management

- Add friend
 - Remove friend
 - View friend list
-

◆ User Discovery

- Search users by name/email
 - Suggest users (basic)
-

4. Architecture (Flow)

User → Request (Add/Search/Profile)



Auth Middleware (verify user)



User Controller



User Service (business logic)



Database (Users / Friends)



Response to client

5. Database Design

You have **2 approaches**:

Option 1 (Recommended): Separate Friend Collection

Users:

```
{
  "userId": "uuid",
  "name": "Avi",
  "email": "avi@email.com"
}
```

Friends:

```
{
  "userId": "A",
  "friendId": "B",
  "createdAt": "timestamp"
}
```

👉 Cleaner and scalable

❌ Option 2 (Avoid for now)

- Store friends inside user document (array)

👉 Harder to scale and query

⚙️ 6. API Endpoints

➤ Get User Profile

GET /api/users/:id

➤ Update Profile

PUT /api/users/update

➤ Add Friend

POST /api/users/add-friend

Request:

```
{
  "friendId": "user_456"
}
```

➤ Remove Friend

DELETE /api/users/remove-friend/:friendId

➤ Get Friend List

GET /api/users/friends

➤ Search Users

GET /api/users/search?q=avi

7. Security Considerations

♦ Authentication Required

- All endpoints must require JWT
-

♦ Prevent Self-Friend

- User cannot add themselves
-

♦ Prevent Duplicate Friends

- Check if already friends before adding
-

♦ Data Exposure Control

- Don't expose sensitive data (password, tokens)
-

8. Edge Cases

- Adding same friend twice
 - Removing non-existing friend
 - Searching empty query
 - Large friend lists (pagination later)
-

9. Design Decisions (for interview)

- Used separate friend collection → scalable
 - Used indexed search → faster queries
 - Decoupled user and relationship data
-



10. Interaction with Other Modules

- **Auth Module** → verifies user
 - **Message Module** → sends messages only to friends
 - **Streak Module** → tracks interactions between friends
-



11. Resume Explanation

Designed a user management system supporting profile handling, friend relationships, and user discovery with optimized database structure and secure access control.



12. Why This Module Matters

- Enables social interaction
- Foundation for messaging
- Shows understanding of relational data

Tab 3

MODULE 3: MESSAGE MODULE (Detailed CORE)

1. Overview

The **Message Module** handles the core functionality of the application:

👉 **sending, receiving, and managing voice messages between users**

This module coordinates with:

- Media Module (audio storage)
 - User Module (sender/receiver)
 - Streak Module (interaction tracking)
 - Ephemeral Module (auto-delete)
-

2. Objectives

- Send voice messages
 - Fetch conversation messages
 - Mark messages as played
 - Maintain message lifecycle
-

3. Responsibilities

◆ Send Message

- Accept sender & receiver
 - Get audio URL (from Media Module)
 - Store metadata in DB
-

◆ Fetch Messages

- Retrieve messages between two users
 - Sort by timestamp
-

♦ **Mark as Played**

- Update message status
 - Trigger ephemeral deletion
-

♦ **Message Validation**

- Ensure users are friends
 - Prevent invalid requests
-

4. Architecture (Flow)

User records audio

- Frontend sends request
 - Backend Message Controller
 - Message Service
 - Calls Media Module (get audio URL)
 - Store metadata in DB
 - Trigger Notification
-

5. Database Schema (Messages)

```
{  
  "messageId": "uuid",  
  "senderId": "user_1",  
  "receiverId": "user_2",  
  "audioUrl": "secure_url",  
  "createdAt": "timestamp",  
  "isPlayed": false,  
  "expiresAt": "timestamp"  
}
```

6. API Endpoints

➤ Send Message

POST /api/messages/send

Request:

```
{  
  "receiverId": "user_2",  
  "audioUrl": "firebase_url"  
}
```

➤ Get Messages (Chat)

GET /api/messages/:userId

👉 Returns conversation between logged-in user & userId

➤ Mark as Played

PUT /api/messages/:messageId/play

👉 Sets `isPlayed = true`

7. Security Design

◆ Authorization Check

- Only sender/receiver can access message
-

◆ Friend Validation

- Only allow messages between friends
-

◆ Secure Audio Access

- Use signed URLs (from Media Module)
-

◆ Input Validation

- Validate:
 - receiverId exists
 - audioUrl format
-

! 8. Edge Cases

- Sending message to non-friend
 - Duplicate message requests
 - Fetching large message history
 - Accessing expired messages
-

↔ 9. Interaction with Other Modules

🎧 Media Module

- Provides audio upload + URL
-

🔥 Streak Module

- On message send → update streak
-

🕒 Ephemeral Module

- On message play → trigger deletion
-

🔔 Notification Module

- Notify receiver
-

10. Design Decisions (for interview)

- Stored metadata separately from media → scalable
 - Used `isPlayed` flag → supports ephemeral logic
 - Used `expiresAt` → supports auto-cleanup
-

11. Performance Considerations

- Index on:
 - senderId
 - receiverId
 - createdAt

 Faster queries for chats

12. Resume Explanation

Developed a scalable messaging system for voice-based communication, handling message lifecycle, secure access, and integration with media storage and real-time interaction modules.

13. Why This Module is CRITICAL

- This is the **heart of your app**
- Most logic + integrations happen here
- Interviewers will focus here

Tab 4

MODULE 4: MEDIA MODULE (Detailed)

1. Overview

The **Media Module** is responsible for handling all audio-related operations, including:

- Uploading voice files
- Generating secure access URLs
- Integrating with storage (Firebase)

👉 It acts as the bridge between your backend and **Firestore Storage**

2. Objectives

- Upload audio files securely
 - Generate time-limited access URLs
 - Avoid exposing public file links
 - Optimize media handling
-

3. Responsibilities

◆ Generate Upload URL

- Backend creates a **signed upload URL**
 - Client uploads directly to Firebase
-

◆ Generate Access URL

- Backend provides **secure, temporary download link**
-

◆ Store Media Metadata

- File size
 - Duration
 - Storage path
-

◆ **Validate Media**

- Check file size
 - Check format (mp3, wav)
-

4. Architecture (Flow)

1. User records audio
 2. App → request upload URL from backend
 3. Backend → generates signed URL
 4. App uploads file to Firebase Storage
 5. Backend stores metadata
 6. Message module uses media URL
-

5. Media Metadata Schema

```
{  
  "mediaId": "uuid",  
  "filePath": "audio/user123/file.mp3",  
  "fileSize": 204800,  
  "duration": 12,  
  "createdAt": "timestamp"  
}
```

6. API Endpoints

➤ **Get Upload URL**

POST /api/media/upload-url

Response:

```
{
```

```
"uploadUrl": "signed_upload_url",  
"filePath": "audio/user123/file.mp3"  
}
```

➤ Get Download URL

GET /api/media/download-url?filePath=xyz

7. Security Design

◆ Signed URLs (VERY IMPORTANT)

- URLs expire after short time (e.g., 5 mins)
 - Prevent unauthorized access
-

◆ No Public Files

- Files should NOT be publicly accessible
-

◆ File Validation

- Max size (e.g., 1–2 MB)
 - Allowed formats only
-

◆ Auth Required

- Only logged-in users can upload/download
-

8. Edge Cases

- Upload failure (network issue)
- Large file uploads
- Invalid file type

- Expired signed URL
-



9. Interaction with Other Modules



Message Module

- Uses media URL when sending message
-



Security Module

- Validates file size/type
-



Ephemeral Module

- Deletes file after message expires
-



10. Design Decisions (for interview)

- Used Firebase Storage for scalability
 - Used signed URLs instead of public links
 - Decoupled media from message metadata
-



11. Performance Considerations

- Upload directly from client → reduces backend load
 - Compress audio before upload
 - Use CDN (Firebase provides it)
-



12. Resume Explanation

Implemented a secure media handling system using Firebase Storage with signed URL-based upload/download, ensuring efficient and protected audio file management.

13. Why This Module Matters

- Handles heavy data (audio files)
- Impacts performance directly
- Shows real-world system design

Tab 5

MODULE 5: STREAK MODULE (Detailed)

1. Overview

The **Streak Module** tracks daily interactions between two users and maintains a **streak count**.

👉 Inspired by Snapchat, this feature is designed to:

- Increase engagement
 - Encourage daily usage
 - Add a “game-like” experience
-

2. Objectives

- Track daily message exchanges
 - Maintain streak count between users
 - Reset streak if interaction breaks
 - Prevent duplicate increments
-

3. Responsibilities

◆ Track Interaction

- Detect when two users exchange messages within a day
-

◆ Increment Streak

- Increase streak if:
 - both users interacted today
 - last interaction was yesterday
-

◆ Reset Streak

- Reset if:
 - no interaction for > 24 hours
-

◆ Prevent Multiple Counts

- Only count **once per day per user pair**
-

4. Architecture (Flow)

User A sends message → Message Module



Trigger Streak Module



Check last interaction



Update streak count



Save in DB

5. Database Schema (Streaks)

```
{
  "user1": "user_A",
  "user2": "user_B",
  "streakCount": 5,
  "lastInteractionDate": "2026-05-01",
  "updatedAt": "timestamp"
}
```

6. API Endpoints

➤ Get Streak

GET /api/streak/:friendId

➤ Update Streak (internal use)

POST /api/streak/update

👉 Usually triggered internally by Message Module

🔒 7. Security Design

- Only authenticated users can access streaks
 - Only involved users can view streak
 - Prevent manual manipulation (no direct increment API for users)
-

⚠️ 8. Edge Cases (VERY IMPORTANT)

! Same Day Multiple Messages

👉 Should NOT increase streak multiple times

! One-Sided Interaction

👉 Decide rule:

Option A (simpler):

- Only sender activity counts

Option B (better):

- Both users must send at least one message

👉 I recommend **Option A for MVP**

! Time Zone Issues

- Use server time (not device time)
- Normalize to UTC

Missed Day

- If gap > 1 day → reset streak
-

9. Streak Logic (Core Algorithm)

```
if (today == lastInteractionDate):  
    do nothing
```

```
else if (today == lastInteractionDate + 1):  
    streakCount += 1
```

```
else:  
    streakCount = 1
```

10. Interaction with Other Modules

Message Module

- Triggers streak update on message send
-

User Module

- Validates friend relationship
-

Notification Module

- Notify: “🔥 Your streak is about to break!”
-

11. Performance Considerations

- Index on user pairs
 - Cache streak data if needed
 - Avoid recalculating frequently
-

12. Resume Explanation

Designed and implemented a streak tracking system to enhance user engagement, handling daily interaction logic, edge cases, and efficient state updates.

13. Why This Module Matters

- Shows product thinking (not just coding)
- Demonstrates logic handling
- Adds uniqueness to your project

Tab 6

MODULE 6: EPHEMERAL MODULE (Detailed)

1. Overview

The **Ephemeral Module** ensures that messages are **temporary** and automatically removed after a certain condition.

👉 This is inspired by apps like Snapchat and is a **key differentiator** in your project.

2. Objectives

- Delete messages after **first play**
 - Delete messages after **time limit (e.g., 24 hours)**
 - Ensure no unauthorized re-access
 - Keep database clean
-

3. Responsibilities

♦ Detect Message Playback

- When user plays a message → mark as played
-

♦ Trigger Deletion

- If `isPlayed = true` → delete message
-

♦ Handle Time-Based Expiry

- Delete messages not played within 24 hours

◆ Remove Media File

- Delete audio from Firebase Storage
-

4. Architecture (Flow)

User plays message

- Message Module updates isPlayed = true
 - Ephemeral Module triggered
 - Delete message from DB
 - Delete audio from Firebase Storage
-

5. Database Fields Used

```
{  
  "messageId": "uuid",  
  "isPlayed": false,  
  "createdAt": "timestamp",  
  "expiresAt": "timestamp"  
}
```

6. Deletion Strategies

◆ Strategy 1: On-Play Deletion (Immediate)

- Triggered when user plays message
- Deletes instantly

👉 Simple and effective

◆ Strategy 2: Scheduled Deletion (Time-Based)

Use:

- Cron job OR background worker

Runs every few minutes:

Find messages where `currentTime > expiresAt`
→ Delete them

♦ Strategy 3 (Best Practice): Hybrid

👉 Use BOTH:

- Immediate deletion on play
 - Scheduled cleanup for missed cases
-

⚙️ 7. API Endpoint

➤ Mark as Played

PUT `/api/messages/:id/play`

👉 This triggers ephemeral logic internally

🔒 8. Security Design

♦ Prevent Replay

- Once `isPlayed = true` → deny access
-

♦ Secure Media Access

- Use short-lived signed URLs
-

◆ Authorization

- Only receiver can mark as played
-

! 9. Edge Cases (VERY IMPORTANT)

! User Closes App Before Deletion

👉 Solution:

- Always run scheduled cleanup
-

! Network Failure During Deletion

👉 Solution:

- Retry mechanism in background job
-

! Audio Still Cached Locally

👉 Solution:

- Clear local cache after playback
-

! Message Access After Expiry

👉 Must return:

```
{  
  "error": "Message expired"  
}
```

🔄 10. Interaction with Other Modules

Message Module

- Updates `isPlayed`
-

Media Module

- Deletes audio file
-

Security Module

- Ensures no unauthorized access
-

11. Design Decisions (for interview)

- Used hybrid deletion strategy for reliability
 - Decoupled message metadata and media cleanup
 - Used background jobs for scalability
-

12. Performance Considerations

- Batch delete expired messages
 - Avoid frequent DB scans (run every few mins)
 - Index `expiresAt` field
-

13. Resume Explanation

Implemented an ephemeral messaging system with hybrid deletion strategy, ensuring automatic cleanup of messages and media using event-driven and scheduled background processes.

14. Why This Module is Important

- Shows system design depth
- Handles lifecycle management
- Demonstrates real-world problem solving

Tab 7

MODULE 7: SECURITY MODULE (Detailed)

1. Overview

The **Security Module** ensures that your application is protected against misuse, unauthorized access, and common attacks.

👉 This is what makes your project look **professional and deployment-ready**, not just functional.

2. Objectives

- Protect APIs from unauthorized access
 - Prevent abuse (spam, flooding)
 - Validate all inputs
 - Secure data transmission
 - Ensure safe media access
-

3. Responsibilities

◆ Authentication Enforcement

- Ensure every protected API uses JWT middleware
-

◆ Input Validation

- Validate all incoming data:
 - userId
 - messageId
 - file size/type
-

◆ Rate Limiting

- Limit number of requests per user/IP
-

◆ Data Sanitization

- Prevent injection attacks
-

◆ Secure Media Access

- Prevent direct public access to audio files
-



4. Architecture (Flow)

Client Request



Security Middleware



JWT Verification



Rate Limiting



Input Validation



Controller → Service → DB



5. Core Security Components

◆ 1. JWT Authentication

- Every request must include:

Authorization: Bearer <token>

- Middleware verifies:
 - token validity
 - expiration
-

♦ 2. Rate Limiting

Example:

- Max 50 requests/min per user

👉 Prevents:

- spam
 - brute force attacks
-

♦ 3. Input Validation

Use libraries like:

- Joi / Zod (Node.js)

Validate:

- Required fields
 - Correct formats
 - Value ranges
-

♦ 4. Data Sanitization

- Remove harmful inputs
 - Prevent:
 - SQL Injection
 - NoSQL Injection
-

♦ 5. Secure Headers

- Enforce HTTPS
 - Use security headers:
 - Content-Security-Policy
 - X-Frame-Options
-

♦ 6. Media Security

- Use **signed URLs** (short-lived)
 - Never expose public file paths
-

6. Example Middleware Flow

request

→ authMiddleware()
→ rateLimiter()
→ validateInput()
→ controller()

7. Common Threats & Handling

Unauthorized Access

 Fix:

- JWT verification
-

Spam / Flooding

 Fix:

- Rate limiting
-

Malicious Input

 Fix:

- Validation + sanitization
-

File Abuse

 Fix:

- File size/type checks
-

Replay Attack (message reuse)

👉 Fix:

- Expiring URLs + ephemeral logic
-

8. Interaction with Other Modules

- **Auth Module** → provides JWT
 - **Message Module** → validates requests
 - **Media Module** → secures file access
 - **Ephemeral Module** → prevents reuse
-

9. Design Decisions (for interview)

- Centralized security middleware
 - Stateless authentication (JWT)
 - Layered security (validation + rate limiting)
 - Zero trust for client inputs
-

10. Performance Considerations

- Use lightweight validation
 - Cache rate limit counters
 - Avoid heavy checks on every request
-

11. Resume Explanation

Implemented a robust security layer including JWT-based authentication, rate limiting, input validation, and secure media access using signed URLs to protect APIs and user data.

12. Why This Module is CRITICAL

- Shows **production readiness**
- Interviewers LOVE security awareness
- Prevents real-world failures

Tab 8

MODULE 8: NOTIFICATION MODULE

(Detailed)

1. Overview

The **Notification Module** is responsible for informing users about important events in real time, such as:

- New voice message received
- Streak about to break
- Friend activity

👉 This module improves **user engagement and retention**, making your app feel alive.

2. Objectives

- Notify users instantly on new messages
 - Remind users about streaks
 - Ensure reliable delivery
 - Avoid notification spam
-

3. Responsibilities

◆ Push Notifications

- Send notification when:
 - new voice snap arrives
-

◆ Streak Alerts

- Notify before streak breaks

◆ Event-Based Triggers

- Trigger notifications from backend events

◆ Token Management

- Store device tokens for each user
-

4. Architecture (Flow)

User A sends message

→ Message Module

→ Notification Module triggered

→ Push service (Firebase Cloud Messaging)

→ User B receives notification

5. Technology Used

Use:

- **Firebase Cloud Messaging (FCM)** from Firebase

👉 Industry-standard for mobile push notifications

6. Database Schema (Device Tokens)

```
{
  "userId": "user_123",
  "deviceToken": "fcm_token_here",
  "updatedAt": "timestamp"
}
```

7. API Endpoints

➤ Register Device Token

POST /api/notifications/register-token

➤ Send Notification (internal)

POST /api/notifications/send

👉 Usually triggered by backend (not frontend)

8. Security Design

◆ Token Ownership

- Ensure token belongs to authenticated user
-

◆ Prevent Abuse

- Do not allow users to trigger notifications directly
-

◆ Rate Limiting

- Avoid notification flooding
-

9. Edge Cases

! User Offline

- FCM queues notification
-

Invalid Token

- Remove expired tokens
-

Multiple Devices

- Store multiple tokens per user
-

Notification Spam

- Debounce repeated events
-

10. Interaction with Other Modules

Message Module

- Trigger on new message
-

Streak Module

- Trigger streak reminder
-

User Module

- Fetch device tokens
-

11. Design Decisions (for interview)

- Used FCM for reliable delivery
- Decoupled notification logic from core modules
- Event-driven architecture (trigger-based)

12. Notification Types

New Message

“ New voice snap received”

Streak Reminder

“ Your streak is about to break!”

Friend Activity (optional)

“Your friend sent a snap”

13. Resume Explanation

Integrated real-time push notifications using Firebase Cloud Messaging, enabling event-driven alerts for messaging and engagement features like streak reminders.

14. Why This Module Matters

- Makes app feel real-time
- Improves engagement
- Shows full-stack capability

Tab 9

FRONTEND MODULE 1: AUTH MODULE (Detailed)

1. Overview

The **Auth Module (Frontend)** handles all user authentication UI and session management.

It is responsible for:

- User login/signup screens
- Communicating with backend auth APIs
- Storing authentication token securely
- Managing user session

👉 This is the **entry point of your app**

2. Objectives

- Allow users to register/login
 - Store JWT securely
 - Maintain login state
 - Redirect user to app after login
-

3. Responsibilities

◆ UI Screens

- Login screen
 - Signup screen
-

◆ API Integration

- Call backend:
 - `/api/auth/login`
 - `/api/auth/register`
-

◆ Token Management

- Store JWT token securely
 - Attach token to every request
-

◆ Session Handling

- Auto-login if token exists
 - Logout functionality
-

4. Architecture (Flow)

User enters credentials

- Auth Screen
 - Network Module (API call)
 - Backend validates
 - JWT token received
 - Store token locally
 - Navigate to Home/Chat screen
-

5. UI Components

◆ Login Screen

- Email input
 - Password input
 - Login button
 - “Go to signup”
-

◆ Signup Screen

- Name
 - Email
 - Password
 - Confirm password
-

◆ Loading State

- Show progress while API call
-

6. Folder Structure

ui/auth/

```
|— LoginScreen.kt
|— SignupScreen.kt
|— AuthViewModel.kt
```

7. ViewModel Responsibilities

- Handle form state
 - Validate inputs
 - Call API
 - Manage loading/error states
-

8. Security Considerations

◆ Secure Token Storage

Use:

- Android Keystore OR Encrypted SharedPreferences

👉 NEVER store token in plain text

◆ Input Validation

- Email format
 - Password length
-

◆ Prevent Multiple Requests

- Disable button during loading
-

! 9. Edge Cases

- Invalid credentials
 - Network failure
 - Empty fields
 - Token expired
-

10. Interaction with Other Modules

Network Module

- Handles API calls
-

Chat Module

- User redirected after login
-

11. Design Decisions (for interview)

- Used MVVM for separation of concerns
- Secure token storage for session safety
- Centralized API handling via network layer



12. UX Considerations

- Show clear error messages
- Smooth navigation after login
- Remember user session



13. Resume Explanation

Developed authentication UI with secure token handling, integrating backend APIs and implementing session persistence using MVVM architecture.



14. Why This Module Matters

- First impression of app
- Controls access to all features
- Ensures smooth onboarding

Tab 10

FRONTEND MODULE 2: VOICE MODULE (Detailed ★ CORE)

1. Overview

The **Voice Module** handles all audio-related operations on the client side:

- Recording voice
- Playing voice messages
- Managing audio files
- Handling permissions

👉 This is the **core interaction layer** of your app

2. Objectives

- Record high-quality audio
 - Play audio smoothly
 - Keep latency low
 - Ensure proper file handling
-

3. Responsibilities

♦ Audio Recording

- Start / stop recording
 - Save audio locally
-

♦ Audio Playback

- Play received audio
- Pause / resume

◆ File Handling

- Store temp audio file
- Prepare for upload

◆ Permission Handling

- Request microphone permission
-

4. Architecture (Flow)

User taps record

- Start recording (MediaRecorder)
 - Save file locally
 - Stop recording
 - Send file to Filter Module / Network Module
-

5. UI Components

◆ Record Button

- Tap to start
 - Tap again to stop
-

◆ Waveform / Timer

- Show recording duration
 - Optional waveform animation
-

◆ Playback Controls

- Play / Pause button
- Progress bar

6. Folder Structure

ui/voice/

```
|— VoiceRecorder.kt
|— VoicePlayer.kt
|— VoiceViewModel.kt
```

7. ViewModel Responsibilities

- Manage recording state
 - Manage playback state
 - Handle file path
 - Expose UI state
-

8. Tools / APIs Used

- MediaRecorder → recording
 - ExoPlayer / MediaPlayer → playback
-

9. Security & Permissions

♦ Microphone Permission

RECORD_AUDIO permission required

♦ Runtime Permission Handling

- Ask user at runtime
 - Handle denial gracefully
-

10. Edge Cases

Permission Denied

 Show message:

“Microphone permission required”

Recording Interrupted

- Handle app going to background
-

Long Recording

- Limit duration (e.g., 20 sec)
-

Corrupt File

- Validate before sending
-

11. Interaction with Other Modules

Filter Module

- Applies effects on recorded audio
-

Network Module

- Uploads audio
-

Chat Module

- Displays playback
-

12. Performance Considerations

- Compress audio before upload
 - Use lightweight format (AAC / MP3)
 - Avoid storing large files locally
-

13. Design Decisions (for interview)

- Local recording to reduce latency
 - Compression before upload
 - Separation of recording & playback logic
-

14. Resume Explanation

Implemented real-time audio recording and playback using Android MediaRecorder and ExoPlayer, optimizing performance with local processing and file compression.

15. Why This Module is CRITICAL

- Core user interaction
- Directly affects UX
- Differentiates your app

Tab 11

FRONTEND MODULE 3: FILTER MODULE (Detailed ★ DIFFERENTIATOR)

1. Overview

The **Filter Module** applies real-time or post-processing effects to recorded audio, such as:

- Robot voice 
- Deep voice 
- Chipmunk 
- Echo / reverb

👉 This is your equivalent of filters in Snapchat, but for voice.

2. Objectives

- Transform recorded audio
 - Provide fun and expressive filters
 - Keep processing fast (low latency)
 - Maintain audio quality
-

3. Responsibilities

♦ Apply Audio Effects

- Modify pitch, speed, or frequency
-

♦ Provide Filter Options UI

- User selects filter before sending
-

◆ **Process Audio File**

- Take recorded file → output filtered file
-

◆ **Return Processed File**

- Pass final audio to Network Module
-

4. Architecture (Flow)

Voice Module → recorded file



Filter Module



Apply effect (pitch/speed/echo)



Generate new audio file



Send to Network Module

5. UI Components

◆ **Filter Selector**

- Horizontal list:
 - Normal
 - Robot
 - Deep
 - Chipmunk
-

◆ **Preview Button**

- Play filtered version before sending
-

◆ **Apply Button**

- Confirm selected filter
-

6. Folder Structure

ui/filters/

```
|— FilterScreen.kt  
|— FilterProcessor.kt  
|— FilterViewModel.kt
```

7. ViewModel Responsibilities

- Track selected filter
 - Manage processing state
 - Provide preview output
-

8. Implementation Approaches

◆ Option 1 (Recommended for MVP)

👉 Simple transformations:

- Speed change → changes pitch
 - Basic audio effects
-

◆ Option 2 (Advanced)

- DSP (Digital Signal Processing)
- External libraries

👉 Do only if time permits

Example Effects Logic

- Chipmunk → increase pitch
 - Deep → decrease pitch
 - Robot → add distortion
-

9. Security Considerations

- Validate file before processing
 - Prevent extremely large files
 - Avoid processing invalid formats
-

10. Edge Cases

Processing Delay

 Show loading indicator

Unsupported Device

 Fallback to original audio

Corrupted Output

 Retry or revert to original

11. Interaction with Other Modules

Voice Module

- Receives recorded audio
-

Network Module

- Sends processed audio
-

Chat Module

- Displays final audio
-

12. Performance Considerations

- Process locally (avoid backend load)
 - Keep processing time < 1–2 sec
 - Use lightweight transformations
-

13. Design Decisions (for interview)

- Client-side processing → reduces backend load
 - Modular filter system → easy to add new filters
 - Trade-off between quality vs speed
-

14. Resume Explanation

Developed client-side audio processing for applying real-time voice filters using lightweight transformations, enhancing user interaction and reducing backend dependency.

15. Why This Module is IMPORTANT

- Makes your app unique
- Shows creativity + technical skill
- High “wow factor” in demo

Tab 12

FRONTEND MODULE 4: CHAT MODULE

(Detailed ★ CORE UI)

1. Overview

The **Chat Module** is responsible for displaying conversations and enabling interaction between users through voice messages.

It handles:

- Chat UI
- Sending messages
- Receiving messages
- Playback controls

👉 This is where all backend + frontend pieces come together.

2. Objectives

- Display conversation between users
 - Allow sending voice messages
 - Play received messages
 - Show message states (played/unplayed)
-

3. Responsibilities

♦ Chat Screen UI

- Show messages in list format
 - Differentiate sender vs receiver
-

♦ Send Message Flow

- Integrate:
 - Voice Module
 - Filter Module
 - Network Module
-

♦ **Receive Messages**

- Fetch messages from backend
 - Update UI
-

♦ **Playback Control**

- Play/pause audio messages
 - Track message state
-

4. Architecture (Flow)

User records audio

- Voice Module
 - Filter Module
 - Network Module (send API)
 - Backend stores message
 - Chat Module refreshes UI
 - Receiver sees message
-

5. UI Components

♦ **Chat Screen Layout**

- Top bar (user name + streak 🔥)
 - Message list
 - Bottom input area (record button)
-

♦ **Message Bubble**

- Left → received
 - Right → sent
-

♦ Audio Message UI

- Play button 
 - Progress bar
 - Duration
-

6. Folder Structure

ui/chat/

```
|— ChatScreen.kt
|— MessageItem.kt
|— ChatViewModel.kt
```

7. ViewModel Responsibilities

- Fetch messages
 - Maintain message list state
 - Handle send message
 - Update UI on new messages
-

8. Message States

Each message should track:

- Sent
- Delivered (optional)
- Played

👉 Display visually:

- Unplayed → highlighted
 - Played → faded
-

9. Security Considerations

- Only allow messages between friends
 - Do not show expired messages
 - Validate message ownership
-

10. Edge Cases

Empty Chat

 Show:

“Start conversation ”

Slow Network

 Show loading indicator

Message Failed to Send

 Retry option

Expired Message

 Show:

“Message expired”

11. Interaction with Other Modules

 Voice Module

- Provides recorded audio
-

Filter Module

- Applies effects
-

Network Module

- Sends & fetches messages
-

Streak Module

- Displays streak
-

12. Performance Considerations

- Lazy loading messages
 - Use RecyclerView / LazyColumn
 - Avoid reloading entire list
-

13. Design Decisions (for interview)

- Separated UI and business logic using MVVM
 - Used lazy loading for performance
 - Integrated multiple modules cleanly
-

14. Resume Explanation

Built a dynamic chat interface for voice-based messaging with playback controls, message state handling, and seamless integration with backend services.

15. Why This Module is CRITICAL

- Main user-facing feature
- Demonstrates UI + system integration
- Key demo component

Tab 13

FRONTEND MODULE 5: STREAK DISPLAY MODULE (Detailed)

1. Overview

The **Streak Module (Frontend)** is responsible for displaying the **streak count** between users and giving visual feedback to maintain engagement.

👉 This is inspired by Snapchat and creates a **habit-forming loop**.

2. Objectives

- Display current streak between users
 - Visually highlight streak importance
 - Show warnings before streak breaks
-

3. Responsibilities

◆ Display Streak Count

- Show 🔥 + number (e.g., 🔥 5)
-

◆ Real-Time Updates

- Update streak after message sent
-

◆ Warning Indicators

- Show alert if streak is about to break

4. Architecture (Flow)

User opens chat

- Chat Module requests streak data
 - Backend returns streakCount
 - Streak Module updates UI
-

5. UI Components

◆ Streak Badge

-  icon + count
 - Display in:
 - Chat header
-

◆ Streak Warning

- Highlight (red/orange glow)
- Message like:

“ Streak ends today!”

◆ Optional Animation

- Small flame animation on increment
-

6. Folder Structure

ui/streak/

```
|— StreakView.kt
|— StreakViewModel.kt
```

7. ViewModel Responsibilities

- Fetch streak data
 - Observe updates
 - Manage UI state
-

8. Data Format

From backend:

```
{  
  "streakCount": 7,  
  "lastInteractionDate": "2026-05-01"  
}
```

9. Security Considerations

- Fetch only for authenticated user
 - Don't expose other users' data unnecessarily
-

10. Edge Cases

No Streak Yet

 Show:

“Start a streak ”

Streak Just Reset

 Show:

“Streak lost ”

Large Streak

👉 Format nicely:

- 🔥 100+
-

11. Interaction with Other Modules

Chat Module

- Displays streak in header
-

Network Module

- Fetches streak from backend
-

12. Performance Considerations

- Fetch once per chat open
 - Avoid frequent API calls
-

13. Design Decisions (for interview)

- Kept logic in backend, UI in frontend
 - Minimal API calls
 - Lightweight UI component
-

14. Resume Explanation

Implemented a streak visualization feature to enhance user engagement, displaying real-time interaction metrics with minimal performance overhead.

15. Why This Module Matters

- Adds **product thinking**
- Improves engagement
- Makes app feel polished

Tab 14

FRONTEND MODULE 6: NETWORK MODULE (Detailed ★ CORE INFRA)

1. Overview

The **Network Module** handles all communication between your Android app and the backend.

It is responsible for:

- Making API calls
- Attaching authentication tokens
- Handling errors
- Managing request/response flow

👉 This module ensures your app talks to your backend **cleanly and securely**

2. Objectives

- Centralize all API calls
 - Automatically attach JWT token
 - Handle errors globally
 - Keep UI independent from networking
-

3. Responsibilities

◆ API Calls

- Send HTTP requests:
 - GET
 - POST
 - PUT
 - DELETE
-

◆ Token Management

- Attach JWT token to every request
-

◆ Error Handling

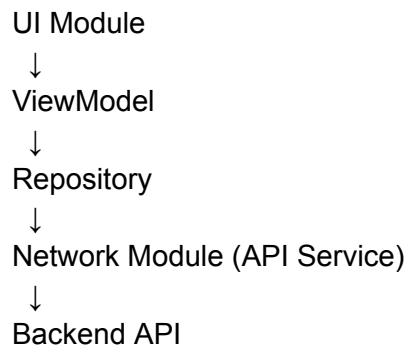
- Handle:
 - network errors
 - server errors
 - unauthorized access
-

◆ Response Parsing

- Convert JSON → Kotlin objects
-



4. Architecture (Flow)



5. Folder Structure

data/network/

- ApiService.kt
- RetrofitClient.kt
- AuthInterceptor.kt
- NetworkResult.kt



6. Tools / Libraries

- Retrofit → API calls
 - OkHttp → networking
 - Gson / Moshi → JSON parsing
-

7. Security Implementation

♦ JWT Interceptor

Automatically attach token:

Authorization: Bearer <token>

👉 Done via interceptor (no manual work in UI)

♦ HTTPS Only

- Ensure all requests use HTTPS
-

♦ Token Expiry Handling

- If 401 error:
 - logout user
 - redirect to login
-

8. API Layer Design

♦ ApiService Example

login()
register()
sendMessage()
getMessages()
getStreak()

♦ Repository Layer

👉 Acts as bridge:

- ViewModel ↔ Network
-

! 9. Error Handling Strategy

! Network Failure

👉 Show:

“No internet connection”

! Server Error

👉 Show:

“Something went wrong”

! Unauthorized (401)

👉 Auto logout

🔄 10. Interaction with Other Modules

🔑 Auth Module

- Sends login/register requests
-

💬 Chat Module

- Fetch/send messages
-

Streak Module

- Fetch streak data
-

Voice Module

- Upload audio
-

11. Performance Considerations

- Use connection pooling (OkHttp handles)
 - Avoid duplicate API calls
 - Cache responses if needed
-

12. Design Decisions (for interview)

- Centralized networking layer
 - Used interceptor for token injection
 - Decoupled UI from API logic
 - Used repository pattern
-

13. Resume Explanation

Built a centralized networking layer using Retrofit and OkHttp with JWT-based authentication, enabling secure and scalable API communication with efficient error handling.

14. Why This Module is CRITICAL

- Connects frontend ↔ backend
- Keeps code clean
- Prevents duplication